Data Science
Department of Mathematics
Faculty of Mathematics and Natural Sciences
University of Indonesia

Prediksi Potensi Nasabah untuk Membuka Deposito dalam Upaya Meningkatkan Efisiensi Marketing dan Profit Penjualan

Eredita Azzahra

Hafizh Abdan

Joe Ifmanur

Muhammad Varendra

Rakyan Adhitya

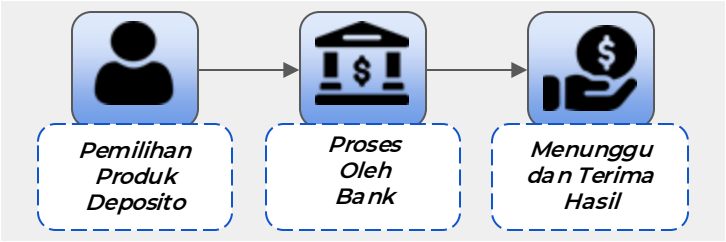
Apa itu Deposito?

*Simpanan di bank dengan **jangka waktu tertentu** yang memberikan bunga lebih tinggi dibandingkan tabungan biasa.*

*Terdapat **beberapa hal** yang perlu diperhatikan...*

- Dana tidak dapat ditarik sebelum jatuh tempo yang telah ditentukan
- Setelah jatuh tempo, dana dan bunganya dapat dicairkan atau diperpanjang
- Mengambil uang dari deposito sebelum jatuh tempo, akan dikenakan penalti seperti penurunan persentase bunga

*Berikut adalah alur **pengajuan deposito**...*



Bank Mendapat Income dari...

- Nasabah melakukan Deposito terhadap Bank
- Pemasukan Deposito akan digunakan untuk memenuhi *demand* pinjaman dari Nasabah lain
- Bunga yang didapat dari pinjaman menjadi penghasilan bagi pihak Bank

Meningkatnya Nasabah yang melakukan Deposito, akan meningkatkan penghasilan dari Bank

*Dengan demikian, **kami akan** melakukan...*

- Identifikasi karakteristik nasabah yang membuka deposito
- Prediksi nasabah yang kemungkinan besar akan membuka deposito

Data Information

Terdapat 21 Fitur pada dataset...

Fitur Kategorik	Fitur Numerik
job	age
marital	duration
education	campaign
default	pdays
housing	previous
loan	emp.var.date
contact	cons.price.idx
month	cons.conf.idx
day of Week	euribor3m
poutcome	nr.employed
y	

Persentase customer yang melakukan deposito sebelumnya...

88%
Customer yang Belum Pernah

12%
Customer yang Pernah

Pengecekan missing value...

Tidak terdapat *Missing Value* pada *Dataset karena* untuk data yang tidak ada nilainya bernilai “unknown”.

Pengecekan duplikasi...

Terdapat 12 data dari fitur secara keseluruhan yang ter-duplikat

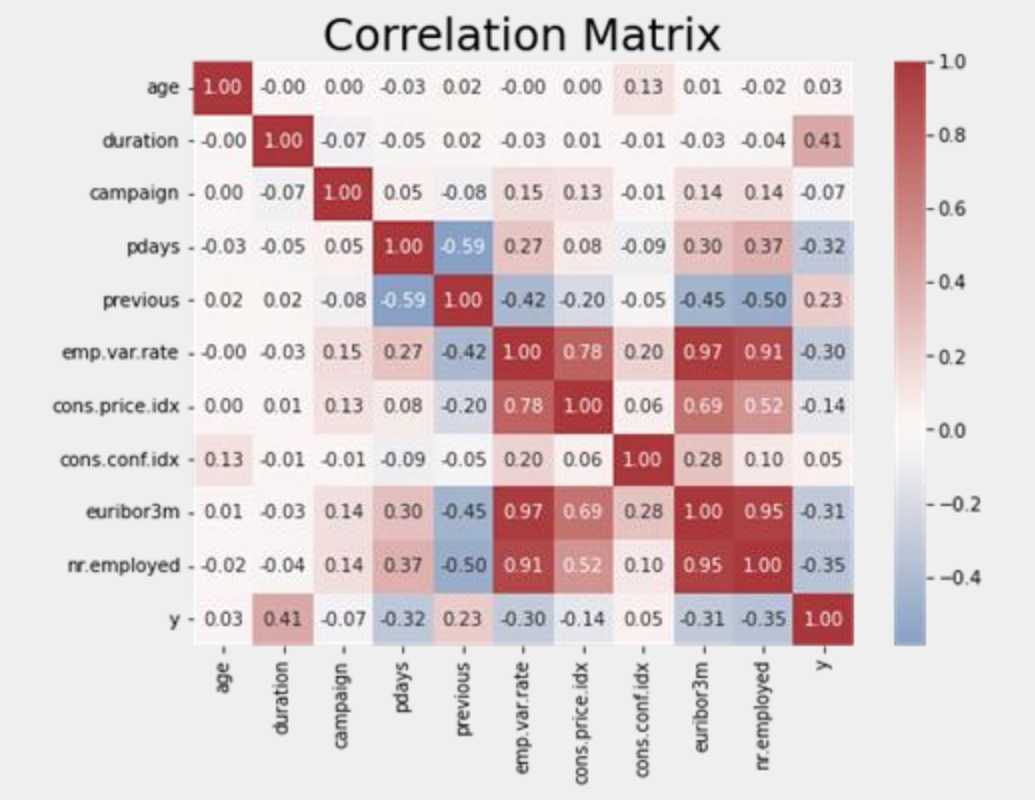
Data tersebut merepresentasikan 0,03% dari keseluruhan data

Apa itu **Matriks Korelasi**?

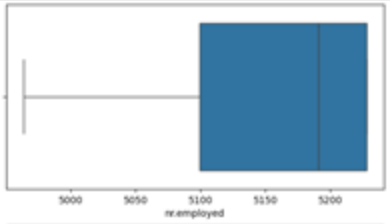
*Representasi matematis dari **hubungan antara variabel** dalam sebuah dataset.*

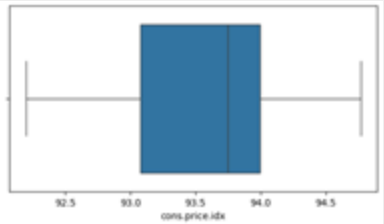
- Mengukur Hubungan Antar Variabel
- Seleksi Fitur
- Pengembangan Hipotesis
- Identifikasi Multikolinearitas

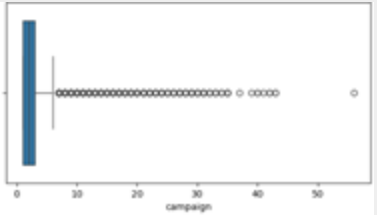
Berikut adalah Matriks Korelasi dari dataset...

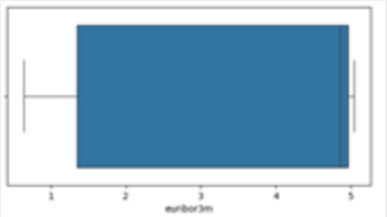


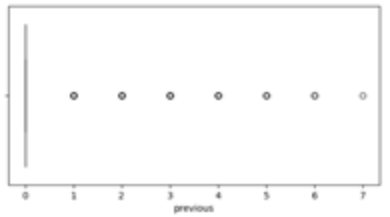
Berikut adalah Outlier dari dataset...

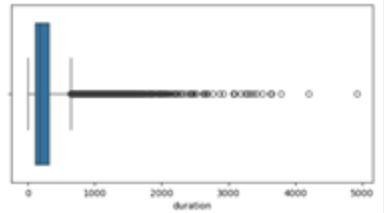


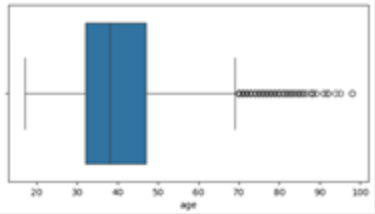


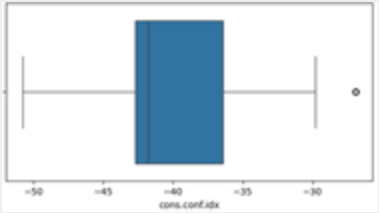


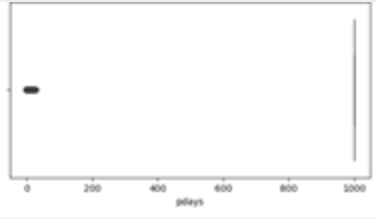


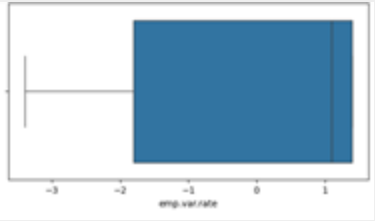




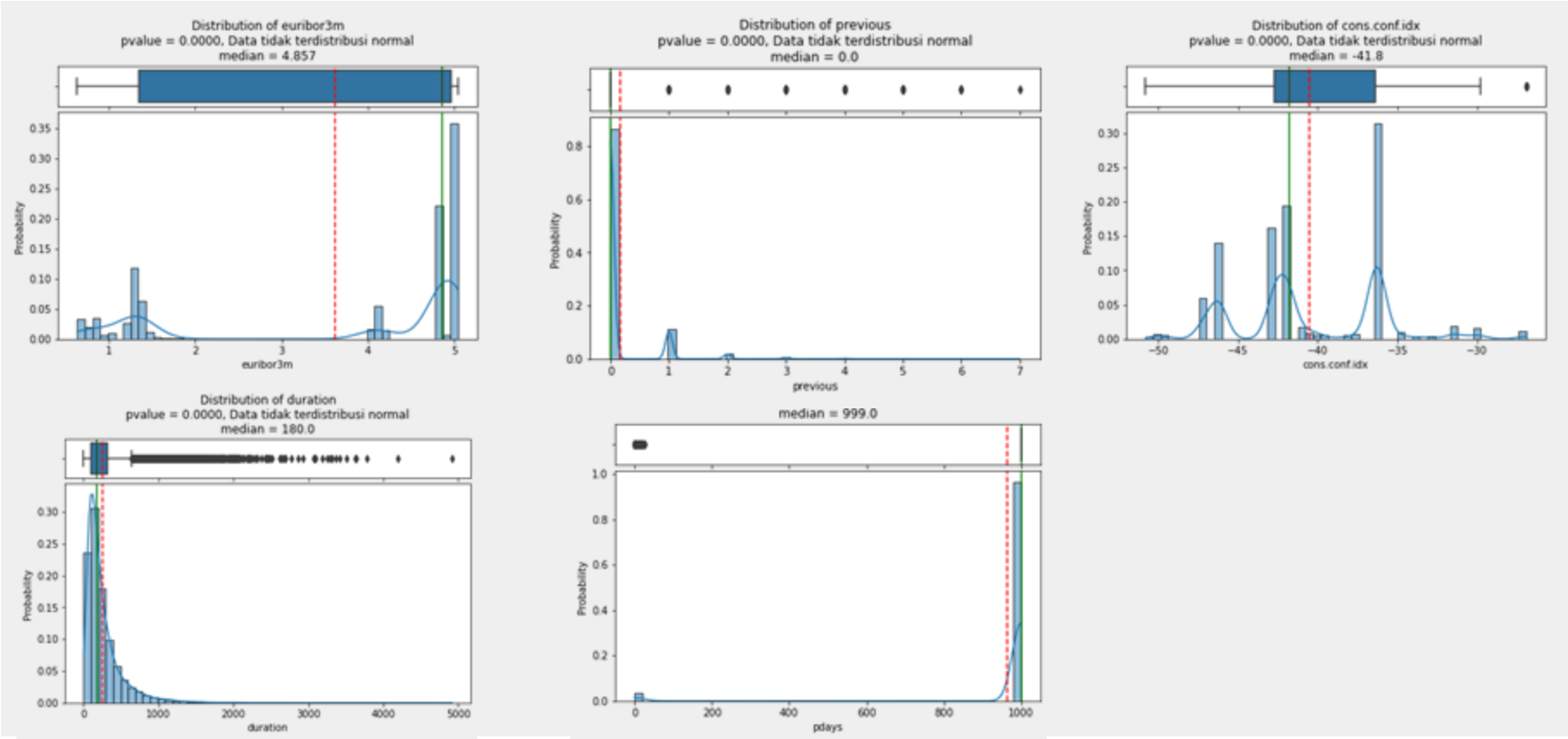




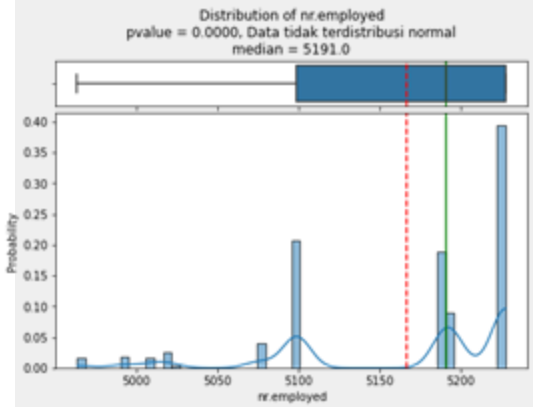


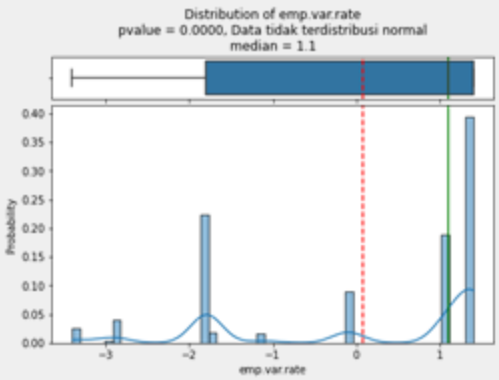


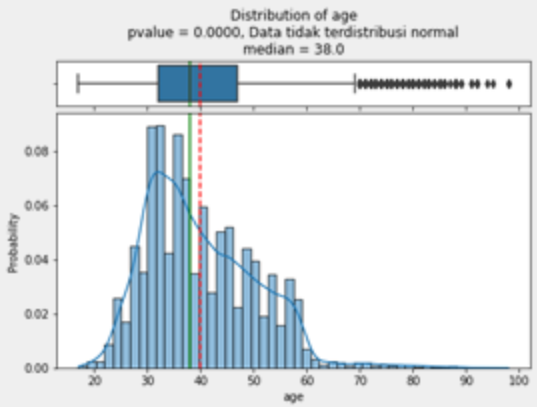
Berikut adalah Distribusi Data Numerikal dari dataset...

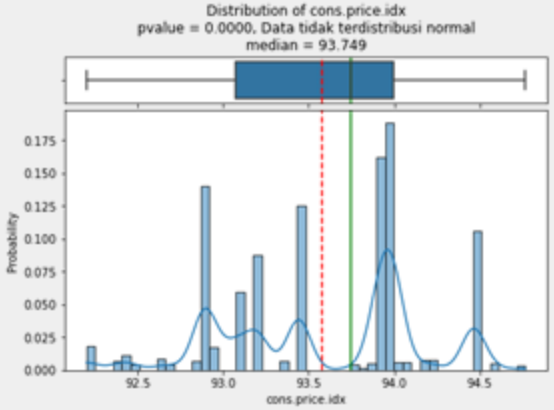


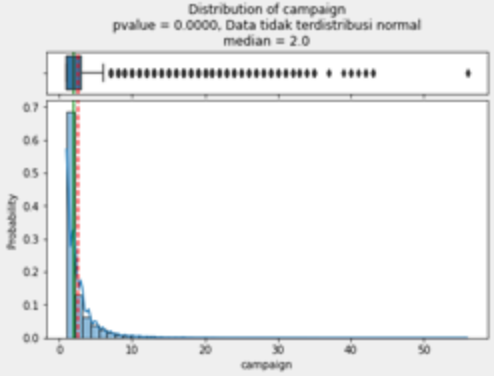
Berikut adalah Distribusi Data Numerikal dari dataset...



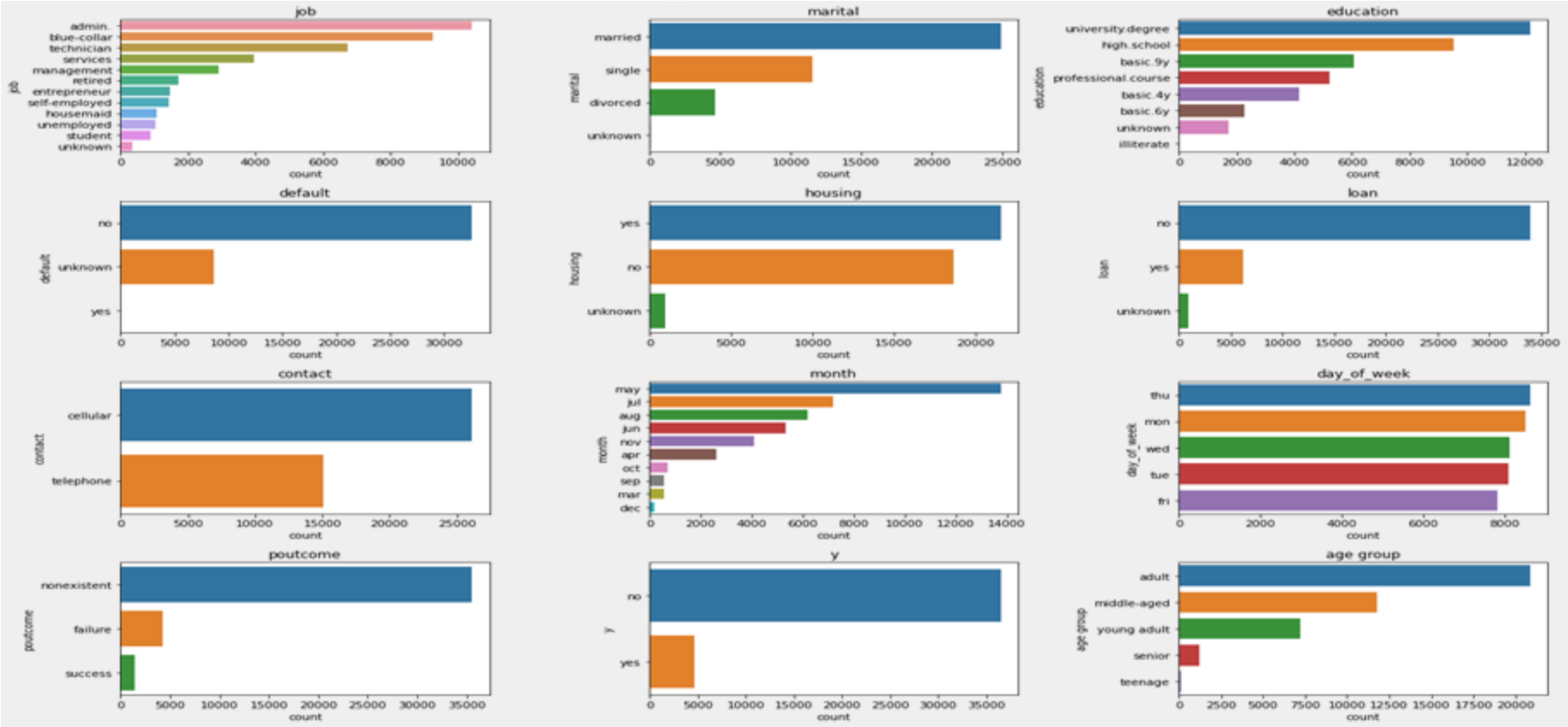








Berikut adalah Distribusi Data Kategorikal dari dataset...



Apa itu **Bining**?

*Sebuah proses **mengelompokkan** data ke dalam bagian-bagian yang lebih kecil yang disebut bin berdasarkan kriteria tertentu.*

- Mengurangi Noise
- Meningkatkan Efisiensi Komputasi
- Memudahkan Visualisasi
- Meningkatkan Akurasi Model

Binning Python

Berikut adalah Bining dalam dataset...

```
[100] def age_group(x):
          if x < 13:
              age_group = 'child'
          elif 13 <= x < 21:
              age_group = 'teenage'
          elif 21 <= x < 31:
              age_group = 'young adult'
          elif 31 <= x < 45:
              age_group = 'adult'
          elif 45 <= x < 60:
              age_group = 'middle-aged'
          else:
              age_group = 'senior'

          return age_group

      df.insert(1, 'age group',value ='')

      df['age group'] = df['age'].apply(age_group)
```

Apa itu **Fill Missing Values**?

*Proses **menggantikan data yang hilang** atau kosong dengan nilai lain.*

- Meningkatkan Akurasi Analisis
- Memperbaiki Kualitas Data
- Mencegah Kesalahan dalam Pemodelan
- Meningkatkan Performansi Algoritma

Berikut adalah Missing Values dari dataset...

```
[190] df = df.replace('unknown',np.nan)
```

```
[191] df.isna().sum()[df.isna().sum()>0]
```

```
job          330
marital       80
education    1730
default      8596
housing       990
loan          990
dtype: int64
```

Berikut adalah Imputasi dari dataset...

	education	job
0	basic.4y	blue-collar
1	basic.6y	blue-collar
2	basic.9y	blue-collar
3	high.school	admin.
4	illiterate	blue-collar
5	professional.course	technician
6	university.degree	admin.

	age group	marital
0	adult	married
1	middle-aged	married
2	senior	married
3	teenage	single
4	young adult	single

	age group	job
0	adult	admin.
1	middle-aged	blue-collar
2	senior	retired
3	teenage	student
4	young adult	admin.

loan	no	yes
age group		
adult	17214	3158
middle-aged	9662	1783
senior	1002	161
teenage	113	23
young adult	5947	1123

housing	no	yes
age group		
adult	9445	10927
middle-aged	5328	6117
senior	520	643
teenage	50	86
young adult	3272	3798

housing	no	yes
marital		
divorced	2092	2398
married	11421	12978
single	5102	6195

loan	no	yes
job		
admin.	8570	1732
blue-collar	7802	1397
entrepreneur	1214	206
housemaid	877	154
management	2414	439
retired	1449	242
self-employed	1186	195
services	3265	601
student	710	142
technician	5613	991
unemployed	838	149

	job	education
0	admin.	university.degree
1	blue-collar	basic.9y
2	entrepreneur	university.degree
3	housemaid	basic.4y
4	management	university.degree
5	retired	basic.4y
6	self-employed	university.degree
7	services	high.school
8	student	high.school
9	technician	professional.course
10	unemployed	university.degree

housing	no	yes
job		
admin.	4678	5624
blue-collar	4401	4798
entrepreneur	641	779
housemaid	491	540
management	1363	1490
retired	786	905
self-employed	641	740
services	1817	2049
student	381	471
technician	2986	3618
unemployed	430	557

loan	no	yes
marital		
divorced	3815	675
married	20615	3784
single	9508	1789

Drop Fitur

Terdapat **dua** fitur yang akan di drop

default

age group

Berikut adalah Drop Fitur dari dataset...

```
df.drop(columns='default', inplace=True)

df.drop(columns='age group', inplace=True)
```

Encoding

Menggunakan dua jenis Encoding
- One Hot Encoding
- Ordinal Encoding

Berikut adalah Code untuk Encoding

Encoding

```
[143] categorical_cols = list(df.select_dtypes(['object','category']).columns)

[144] # menampilkan jumlah unik dan unique values untuk setiap kolom
      pd.set_option('display.max_colwidth', None)
      # data uni di tiap kolom
      listItem = []
      for col in categorical_cols[:-1] :
          listItem.append( [col, df[col].nunique(), df.sort_values(col)[col].unique()])

      tabel1Desc = pd.DataFrame(columns=['Column Name', 'Number of Unique', 'Unique Values'],
                                data=listItem)
      tabel1Desc
```

Berikut adalah Code Mapping Ordinal Encoder dan Transformasi

```
[145] # mapping untuk ordinal encoder

      ordinal_mapping = [
          {'col':'education', 'mapping':{None:0, 'illiterate':1, 'basic.4y':2, 'basic.6y':3, 'basic.9y':4, 'high.school':5,
          {'col':'poutcome', 'mapping':{None:0, 'success':2, 'failure':1, 'nonexistent':0}}
      ]

[146] transformer = ColumnTransformer([
          ('one_hot_encoder', OneHotEncoder(drop='first'), ['job', 'marital', 'housing', 'loan', 'contact', 'month', 'day_of_
          ('ordinal', ce.OrdinalEncoder(cols=['education','poutcome'], mapping=ordinal_mapping), ['education', 'poutcome']),
      ], remainder='passthrough')
```

Data Imbalance

Kondisi di mana jumlah sampel dalam kelas yang berbeda dari suatu dataset tidak seimbang

Berikut adalah Code untuk Data Imbalance

Data Imbalance

```
[148] df['y'].value_counts() / len(df)

y
no     0.887337
yes    0.112663
Name: count, dtype: float64
```

*Dataset **tidak seimbang** karena proporsi kelas minoritas sebesar 11.26%*

Akan digunakan Metode SMOTENC

Synthetic Minority Over-sampling Technique for Nominal and Continuous

Menghindari overfitting karena penduplikasian data yang telah ada sebelumnya

Menambah jumlah sampel dalam kelas minoritas hingga seimbang dengan kelas mayoritas

Berikut adalah Code untuk SMOTENC

```
[150] # define oversampling
      resample = SMOTENC(random_state=0, categorical_features=[3, 12]) #['education', 'poutcome']
```

Change Value

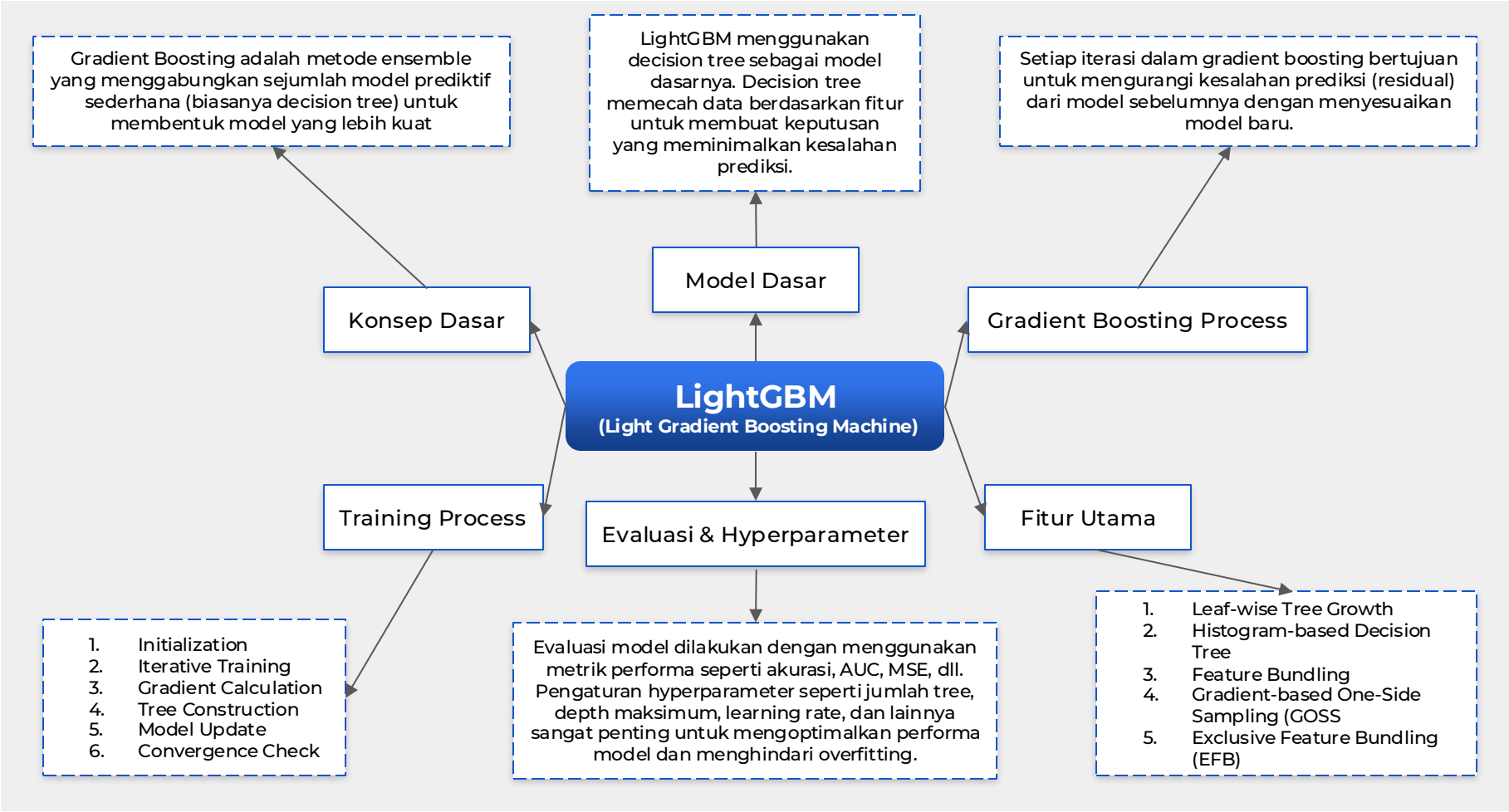
Kita ubah terlebih dahulu nilai pada kolom y dari 'yes' dan 'no' menjadi 1 dan 0. Karena fokus utama kita adalah yang memesan deposit, maka :

Target :

0 : Tidak memesan deposito (y == 'no')

1 : Memesan deposito (y == 'yes')

Method


Gradient Boosting adalah metode ensemble yang menggabungkan sejumlah model prediktif sederhana (biasanya decision tree) untuk membentuk model yang lebih kuat
LightGBM menggunakan decision tree sebagai model dasarnya. Decision tree memecah data berdasarkan fitur untuk membuat keputusan yang meminimalkan kesalahan prediksi.
Setiap iterasi dalam gradient boosting bertujuan untuk mengurangi kesalahan prediksi (residual) dari model sebelumnya dengan menyesuaikan model baru.
Konsep Dasar
Model Dasar
Gradient Boosting Process
LightGBM
(Light Gradient Boosting Machine)
Training Process
Evaluasi & Hyperparameter
Fitur Utama
1. Initialization
2. Iterative Training
3. Gradient Calculation
4. Tree Construction
5. Model Update
6. Convergence Check
Evaluasi model dilakukan dengan menggunakan metrik performa seperti akurasi, AUC, MSE, dll. Pengaturan hyperparameter seperti jumlah tree, depth maksimum, learning rate, dan lainnya sangat penting untuk mengoptimalkan performa model dan menghindari overfitting.
1. Leaf-wise Tree Growth
2. Histogram-based Decision Tree
3. Feature Bundling
4. Gradient-based One-Side Sampling (GOSS
5. Exclusive Feature Bundling (EFB)

Split Data Training & Testing

Mendefinisikan *variabel fitur dan variabel target.*

X: Variabel Fitur

Y: Variabel Target

Berikut adalah Split Data Training & Testing...

```
[67] X = df.drop('y', axis=1)
     y = df['y']
```

```
[68] X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0, stratify=y)
```

Apa itu **Model Benchmarking**?

Mengevaluasi dan **membandingkan** algoritma machine learning berdasarkan kemampuannya dalam mempelajari data.

Metrik yang digunakan adalah **recall**, karena kesalahan false negative memberikan biaya yang jauh lebih besar dibandingkan false positive.

Menentukan model terbaik berdasarkan recall yang **paling optimal**.

Berikut adalah Output Benchmark Model dengan Over-Sampling...

	model	mean recall	mean precision	fit time	score time
0	Logistic Regression	0.629	0.336	5.282	0.043
1	KNN	0.510	0.306	3.787	2.687
2	Decision Tree	0.509	0.461	5.047	0.041
4	AdaBoost	0.454	0.457	7.419	0.167
5	Gradient Boosting	0.400	0.515	17.011	0.049
3	Random Forest	0.330	0.508	11.814	0.199
6	Light GBM	0.320	0.601	5.235	0.097
7	Xgboost	0.313	0.583	5.985	0.061

Apa itu **Tuning Hyperparameter**?

Proses **penyesuaian** dan **pemilihan** nilai optimal untuk hyperparameter suatu model machine learning guna meningkatkan kinerja model tersebut

- Mengurangi Overfitting dan Underfitting
- Meningkatkan Akurasi Model
- Mengidentifikasi Model Terbaik
- Optimisasi Waktu Pelatihan

Berikut adalah Tuning Hyperparameter dari dataset...

	model	best param	best recall
6	Light GBM	{'algo__scale_pos_weight': 30}	0.870074
7	Xgboost	{'algo__scale_pos_weight': 30}	0.758263
0	Logistic Regression	{'algo__solver': 'saga'}	0.692152
3	Random Forest	{'algo__max_depth': 3}	0.645877
4	AdaBoost	{'algo__learning_rate': 0.1}	0.575165
1	KNN	{'algo__n_neighbors': 21}	0.569699
5	Gradient Boosting	{'algo__learning_rate': 0.01}	0.567978
2	Decision Tree	{'algo__max_depth': 3}	0.564531

Cross Validation Benchmark Model 2

Without Oversampling...

	model	mean recall	mean precision	fit time	score time
6	Light GBM	0.861	0.161	0.661	0.104
7	Xgboost	0.754	0.187	0.584	0.056
1	KNN	0.249	0.618	0.248	2.180
0	Logistic Regression	0.228	0.660	0.660	0.071
2	Decision Tree	0.189	0.702	0.159	0.037
4	AdaBoost	0.184	0.706	1.740	0.109
3	Random Forest	0.168	0.731	1.542	0.105
5	Gradient Boosting	0.128	0.759	4.915	0.053

With Oversampling...

	model	mean recall	mean precision	fit time	score time
6	Light GBM	0.870	0.153	5.226	0.096
7	Xgboost	0.758	0.175	6.108	0.053
3	Random Forest	0.646	0.319	6.257	0.079
0	Logistic Regression	0.629	0.336	4.854	0.048
4	AdaBoost	0.575	0.346	7.090	0.088
1	KNN	0.570	0.333	4.017	2.633
5	Gradient Boosting	0.568	0.419	17.500	0.053
2	Decision Tree	0.565	0.371	3.952	0.044

Berdasarkan hasil cross-validation dengan dan tanpa oversampling...

Penggunaan oversampling membutuhkan **waktu yang lebih lama** untuk training (fit time).

Nilai **recall dan precision** LGBM dan XGBoost **tidak menunjukkan perbedaan signifikan** antara menggunakan oversampling atau tidak.

Kami memutuskan untuk **tidak menggunakan oversampling** dalam model.

LGBM Classifier
Hyperparameter Tuning

Parameter yang akan di-tuning

max_bin = max number of bins that feature values will be bucketed in.

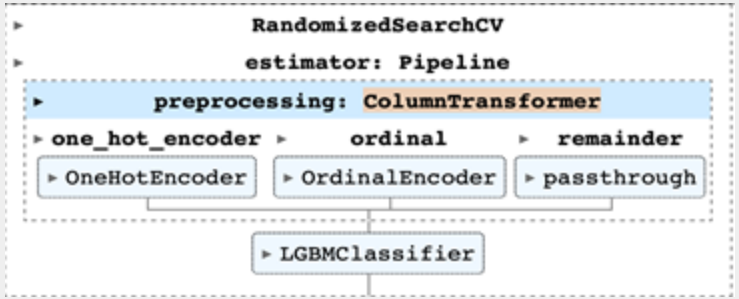
n_estimators = number of boosting iterations. Increasing this number improves the optimality of splits at the cost of higher computation time.

learning rate = shrinkage rate to prevent overfitting.

num_leaves (min_split_loss) = max number of leaves in one tree

scale_pos_weight = weight of labels with positive class

Berikut adalah Hyperparameter Tuning LGBM Classifier dengan RandomizedSearchCV...




```
[ ] lgbm_tuning.best_params_

{'algo__scale_pos_weight': 30,
 'algo__num_leaves': 51,
 'algo__n_estimators': 100,
 'algo__max_bin': 300,
 'algo__learning_rate': 0.03}

[ ] lgbm_tuning.best_score_

0.875253865872819
```

XGBoost
Hyperparameter Tuning

Parameter yang akan di-tuning

max_bin: Jumlah maksimum bin diskrit untuk mengelompokkan fitur kontinu.

n_estimators: Jumlah iterasi yang akan dicoba oleh XGBoost untuk pembelajaran.

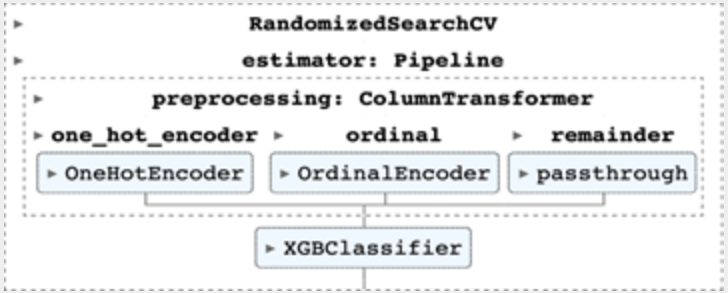
eta (learning rate): Ukuran langkah penyusutan yang digunakan dalam pembaruan untuk mencegah overfitting.

gamma (min_split_loss): Pengurangan kerugian minimum yang diperlukan untuk membuat partisi lebih lanjut pada node daun pohon.

max_depth: Kedalaman maksimum tree.

scale_pos_weight: Mengontrol keseimbangan bobot positif dan negatif.

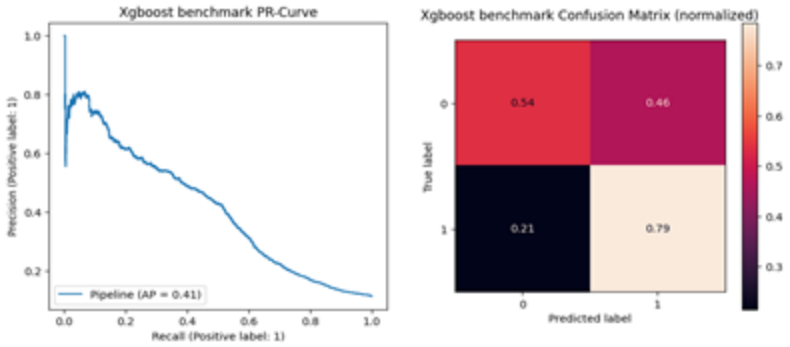
Berikut adalah Hyperparameter Tuning XGBoost dengan RandomizedSearchCV...

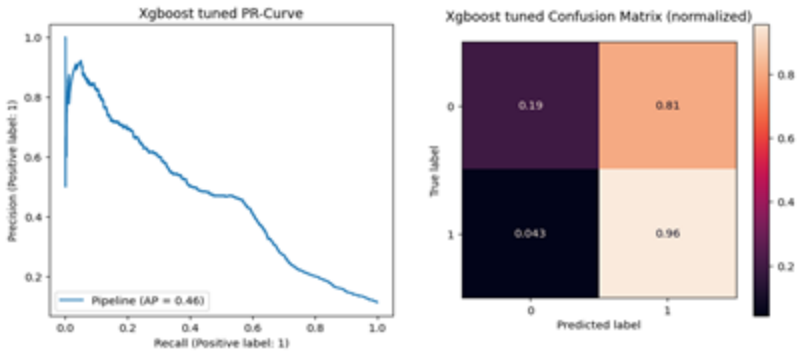



```
[ ] xgboost_tuning.best_params_

{'algo__scale_pos_weight': 30,
 'algo__n_estimators': 100,
 'algo__max_depth': 5,
 'algo__max_bin': 255,
 'algo__gamma': 0.1,
 'algo__eta': 0.05}

[ ] xgboost_tuning.best_score_

0.943089804018854
```

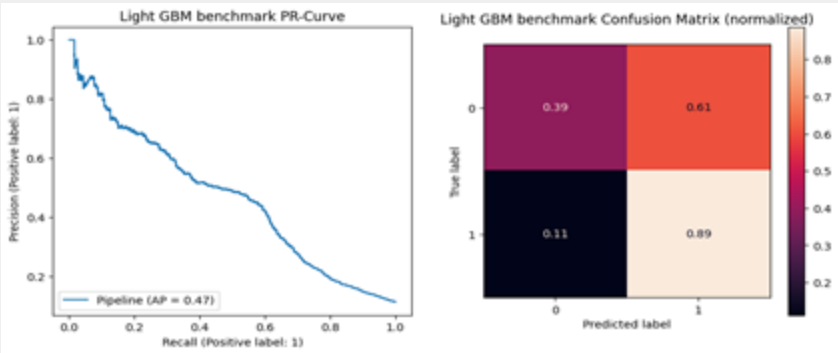
Perbandingan Benchmark dan Hasil Tuned

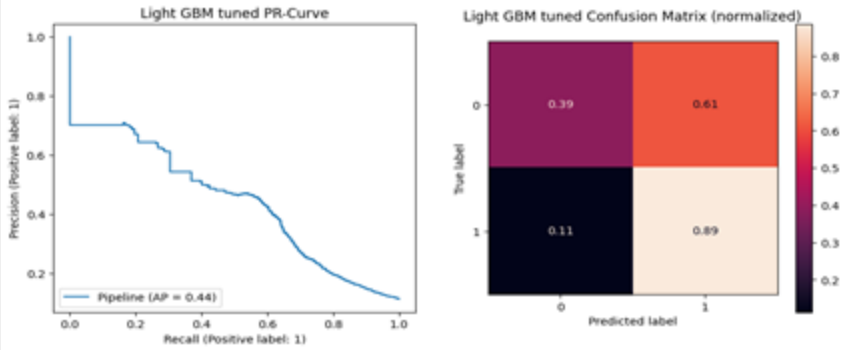
XGBoost





LGBM Classifier





Perbandingan Benchmark dan Hasil Tuned

Berikut adalah perbandingan spesifiknya...

	model	recall	precision	false positive rate	profit	fit time	score time
3	Light GBM benchmark	0.888	0.156	0.61	23223	0.583	0.102
0	Light GBM tuned	0.887	0.155	0.61	23124	0.750	0.131
2	Xgboost tuned	0.957	0.131	0.81	23051	0.603	0.063
1	Xgboost benchmark	0.785	0.177	0.46	21511	0.639	0.077

***XGBoost** memiliki recall tuned lebih tinggi dari LGBM.*

***XGBoost** memiliki False Positive Rate (FPR) lebih tinggi dari LGBM.*

***LGBM** akan digunakan sebagai model utama.*

FPR yang tinggi dapat berdampak buruk bagi model dalam memprediksi.

Menentukan Fitur yang Menjadi Final Modeling

***Berikut adalah feature selection* pada dataset...**

	Thresh	Number of Features	recall	precision	false positive rate	profit	fit time	score time
0	0	39	0.888	0.156	0.61	23223	1.037	0.101
1	15	32	0.882	0.158	0.60	23139	1.051	0.101
3	50	15	0.888	0.152	0.63	22962	0.960	0.095
2	30	22	0.878	0.155	0.61	22904	0.978	0.105
4	70	9	0.898	0.145	0.68	22703	0.934	0.090
5	100	5	0.903	0.141	0.70	22521	1.007	0.126
6	400	2	0.936	0.130	0.80	22446	1.157	0.124

Dapat dilihat bahwa semakin banyak jumlah fitur maka optimisasi profit bisnis (*business score*) semakin banyak.

Fitur yang tersisa tetap digunakan agar *business score* tetap berjalan baik untuk menghasilkan profit yang besar dan mengefisiensikan biaya marketing.

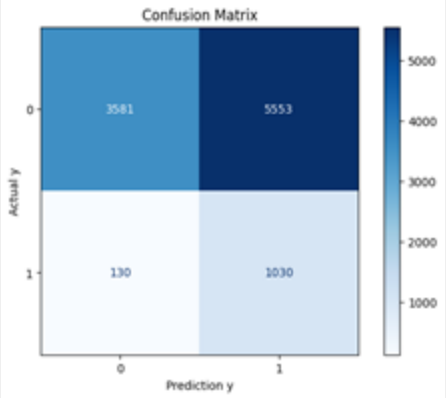
Final Model & Confusion Matrix

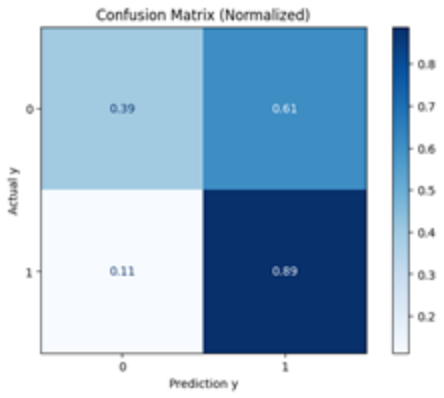
Didapat recall sebesar 89%, maka akan digunakan recall sebagai prediksi akhir.

```
[258] y_pred = final_model.predict(X_test)
      print(classification_report(y_test, y_pred))

              precision    recall  f1-score   support

           0       0.96      0.39      0.56      9134
           1       0.16      0.89      0.27      1160

    accuracy                           0.45     10294
   macro avg       0.56      0.64      0.41     10294
weighted avg       0.87      0.45      0.52     10294
```

Menggunakan variabel dummy untuk memprediksi potensi nasabah.

Nasabah dengan kriteria tersebut berpotensi sebesar 94% untuk tidak mengambil deposito pada bank.





	True	False
Positive	1.030 Orang	5.553 Orang
Negative	3.581 Orang	130 Orang

False Positive Rate

61%

False Negative Rate

0,11%

Conclusion

Tanpa Machine Learning

*Tanpa menggunakan machine learning, kita **tidak dapat memprediksi** nasabah yang akan melakukan deposit atau tidak*

Kita harus menawarkan produk dan menelepon semua nasabah.

Bagaimana Perhitungannya...

Jumlah nasabah untuk dasar perhitungan

Nasabah yang ditelepon adalah **seluruh nasabah** sebanyak 10.294 orang

Nasabah yang **aktualnya** melakukan deposit sebanyak 1.160 orang

Pendapatan - Biaya = (Jumlah nasabah yang deposit x Pendapatan per deposit) - (Jumlah nasabah yang ditelepon x Biaya telemarketing)

(1.160 x 31,75 Euro) - (10.294 x 1,44 Euro) → ***36.830 - 14.824***

22.006 Euro

Dengan Machine Learning

Jumlah nasabah untuk dasar perhitungan

Nasabah yang ditelepon adalah nasabah yang **diprediksi** akan deposit

TP + FP = 1.030 + 5.553
= 6.583 orang

Nasabah yang **aktualnya** melakukan deposit sebanyak 1.030 orang

TP = 1.030 orang

Pendapatan - Biaya = (Jumlah nasabah yang deposit x Pendapatan per deposit) - (Jumlah nasabah yang ditelepon x Biaya telemarketing)

(1.030 x 31,75 Euro) - (6.583 x 1,44 Euro) → ***32.072 - 9.479***

23.222 Euro

*Peningkatan Keuntungan sebesar **1.216 Euro***

*Persentase Keuntungan sebesar **5,5%***

Daftar Pustaka

- CEICdata.com. (2018, July 13). *Portugal PT: Deposit Accounts: per 1000 Adults: Commercial Banks*. Economic Indicators | CEIC. https://www.ceicdata.com/en/portugal/banking-indicators/pt-deposit-accounts-per-1000-adults-commercial-banks

- Arthana, R. (2021, December 9). Mengenal Accuracy, Precision, Recall dan Specificity serta yang diprioritaskan dalam Machine Learning. *Medium*. https://rey1024.medium.com/mengenal-accuracy-precission-recall-dan-specificity-serta-yang-diprioritaskan-b79ff4d77de8

- Menghani, D. (2024, June 6). Cross-Validation with XGBoost — Enhancing Customer Churn Classification with Tidymodels. *Medium*. https://towardsdatascience.com/cross-validation-with-xgboost-enhancing-customer-churn-classification-with-tidymodels-fd9e1bc8696d?source=collection_home---4------1----------

- *Welcome to LightGBM's documentation! — LightGBM 4.3.0.99 documentation*. (n.d.). https://lightgbm.readthedocs.io/en/latest/index.html

- https://take-profit.org/en/statistics/interest-rate/portugal/

Thank You!